A Digital FP CIM Macro With Cascaded Row-Elimination for Exponent Comparison and In-Memory Mantissa Sparsity Detection

Wenjuan Lu, *Member, IEEE*, Hao Liu, Kang Meng, Xiaobo Gong, Zhiqiang Wang, Wei Hu, Chunyu Peng, *Member, IEEE*, Shan Gao, Zhiting Lin, *Senior Member, IEEE*, and Xiulong Wu, *Senior Member, IEEE*

***Abstract*—Floating-point (FP) computing-in-memory (CIM) addresses the energy efficiency bottleneck of von Neumann architectures and fixed-point CIM in high-accuracy neural network training/inference. However, existing FP CIM designs still suffer from limited exponent-path parallelism and low mantissa-path energy efficiency. This article proposes a novel FP CIM architecture using: 1) a cascaded row-elimination comparison mechanism for single-cycle, high-parallelism exponent max-value comparison; 2) an in-memory sparse feature detection method that skips redundant mantissa computing modules based on different input/weight mantissa patterns to reduce energy consumption; and 3) separated exponent/mantissa CIM modules enabling pipelining and a mantissa bit-extension strategy optimizing accuracy, area, and energy efficiency. Simulation results of a 28-nm 19-kb macro show a 496-MHz operating frequency at 0.9 V and a peak energy efficiency of 12.88 TFLOPS/W.**

***Index Terms*—Computing-in-memory (CIM), energy efficiency optimization, floating point (FP), sparse feature detection, static random access memory (SRAM).**

I. Introduction

With the exponential growth in both the scale and complexity of artificial intelligence (AI) models, traditional hardware systems based on the von Neumann architecture face significant energy efficiency challenges primarily attributed to frequent data transfer operations. The computing-in-memory (CIM) architecture eliminates bottlenecks by integrating multiply–accumulate (MAC) operations into memory, such as static random access memory (SRAM) [1], [2], [3] and resistive RAM (ReRAM) [4], [5], [6]. It significantly reduces the overhead of data movement and has become the core direction of energy-efficient AI accelerators. Thanks to its mature technology, SRAM-based CIM becomes the most stable choice compared to nonvolatile memory-based CIM (nvCIM).

Received 16 July 2025; revised 11 September 2025; accepted 4 October 2025. Date of publication 23 October 2025; date of current version 30 December 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62471003 and Grant 62274001, in part by the University Synergy Innovation Program of Anhui Province under Grant GXXT-2023-003 and Grant GXXT-2023-011, and in part by the National Natural Science Foundation of the Higher Education Institutions of Anhui Province under Grant 2023AH040011. *(Corresponding author: Wenjuan Lu.)*

The authors are with the School of Integrated Circuits, Anhui University, Hefei 230039, China (e-mail: wjlu@ahu.edu.cn).

Digital Object Identifier 10.1109/TVLSI.2025.3619502

Based on distinct computational paradigms, SRAM-CIM technology can be divided into analog- and digital-domain implementation paths, which exhibit significant complementary characteristics in terms of accuracy, energy efficiency, and flexibility. Analog-domain CIM (such as SRAM-based current/charge/time accumulation schemes [7], [8], [9]) shows excellent energy efficiency in integer (INT) calculations by directly implementing MAC operations with analog signals. However, due to nonideal effects such as process variation and noise interference, high-accuracy floating-point (FP) operations are inherently unreliable or imprecise in analog signal processing. Furthermore, an inflexible analog data path precludes the execution of FP operations altogether, limiting functionality to INT MAC, which is insufficient for the high-accuracy inference and training requirements of modern neural networks [10]. Digital-domain CIM (such as all-digital SRAM-CIM [11]) implements MAC operations using Boolean logic and adder trees. Although slightly less energy-efficient than analog schemes, it offers greater flexibility and minimal accuracy loss.

Digital signals are less affected by nonideal effects and support high-accuracy FP operations. Lee et al. [12] proposed an FP architecture using exponent CIM and mantissa digital core processing. This design demonstrates significant energy efficiency improvement in exponent operations. However, compared to the traditional FP CIM, the non-CIM-based mantissa calculation module demonstrates no significant improvement in energy efficiency. Jeong et al. [13] designed a digital CIM architecture that supports dynamically configurable precision from FP8 to FP32, which implements exponent and mantissa calculations using near-memory computing units, significantly reducing area consumption. However, this design only supports single-bit data processing per cycle, resulting in a notable bottleneck in computational throughput. Tu et al. [14] proposed an ReCIM architecture that implements an FP CIM processor with exponent alignment-free and mantissa Booth encoding. This architecture eliminates traditional exponent addition logic by prealigning the exponent, while the mantissa Booth encoding reduces operation cycles. However, an independent peripheral encoding circuit for the mantissa constrains optimization of energy efficiency. Yan et al. [15] proposed

1063-8210 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:57 UTC from IEEE Xplore. Restrictions apply.

an energy-efficient FP CIM processor utilizing the long-tail distribution of exponents. To achieve high energy efficiency, this processor separates FP computations into dense and sparse components. The dense components are processed by an optimized fixed-point CIM core using an FP-to-INT conversion workflow. Meanwhile, a programmable digital core handles the sparse components. However, it relies on external circuits to realize FP-to-INT conversion, which introduces additional hardware overhead and potential data transmission bottlenecks.

Overall, the structural characteristics of FP (specifically its three-field structure consisting of sign, exponent, and mantissa) pose multiple challenges to the efficient implementation of MAC operations in CIM architecture.

1) Parallelism bottleneck in the exponent path. Before performing mantissa operations, FP MAC necessitates exponent alignment, which lies in the detection of multibit exponent maximum values. Existing schemes mostly use iterative comparison or binary search methods, resulting in the exponent alignment cycle growing linearly with the exponent bit width, severely restricting the throughput of CIM. 2) Energy efficiency loss in the mantissa path. Mantissa MAC accounts for more than 70% of the energy consumption in FP operations [14]. Traditional CIM schemes, to fit FP formats, typically convert the mantissa into long-bit-width fixed-point formats or rely on external digital circuits for alignment. The former requires multicycle operations, while the latter leads to energy efficiency loss due to the data round-trip to the external circuit. Furthermore, the inherent sparsity in mantissa operations remains underutilized, leading to substantial redundant computations. 3) Imbalances in accuracy, area, and energy efficiency in existing FP CIM architectures. Typical solutions adopt exponent prealignment to eliminate exponent addition for area reduction, but this results in significant accuracy loss. Although multicycle serial processing through the near-memory computing unit can reduce the hardware overhead, the constraint of the single-cycle single-bit data path leads to the attenuation of the computing parallelism.

To address the above challenges, this article proposes a novel FP CIM circuit for the BF16 format, which achieves multibit exponent parallel comparison, mantissa sparse feature detection processing, and redundancy skipping. The main contributions include the following.

1) Based on a cascaded row-elimination comparison mechanism, it supports parallel multirow exponent comparison. This enables progressive filtering across columns from the most significant bit (MSB) to the least significant bit (LSB) in a single cycle, achieving fast and highly parallel exponent maximum value comparison.

2) This article presents an in-memory mantissa sparse feature detection method. Sparse feature detection circuits embedded in CIM bitcells perform predetermination of multimodal sparsity in input/weight mantissas before Booth encoding. When a sparse pattern is identified, the corresponding computing module is disabled to avoid redundant energy consumption. Conversely, the full computation path remains active to guarantee accuracy.

3) An FP pipelined processing approach is implemented using a dual-CIM architecture. By separating the exponent and mantissa CIM modules, it enables pipelined operation. Part of the exponent and mantissa operations are integrated in the memory arrays, reducing frequent data movement and further improving energy efficiency. Moreover, an appropriate mantissa bit-extension strategy is also adopted to balance accuracy, area, and energy efficiency.

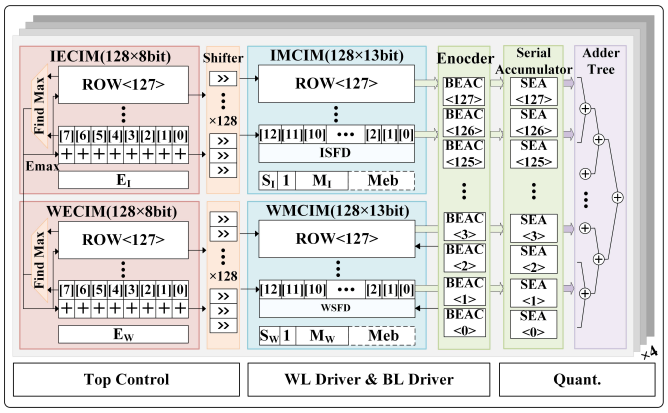


Fig. 1. Architecture of the proposed FP CIM macro.

The remainder of this article is structured as follows. Section II provides an overview of the proposed FP CIM macro and elaborates on its operational principles. Section III presents the results of the performance evaluation. Finally, Section IV concludes this article.

II. Proposed FP CIM Architecture

A. Overall Architecture

The overall architecture of the proposed FP CIM macro is illustrated in Fig. 1. This macro integrates four 1-kb input exponent CIM (IECIM) SRAM arrays, four 1-kb weight exponent CIM (WECIM) SRAM arrays, four 1.375-kb input mantissa CIM (IMCIM) SRAM arrays, and four 1.375-kb weight mantissa CIM (WMCIM) SRAM arrays. Additionally, it incorporates four shifters, four encoders, four serial accumulators, and four adder trees.

Exponents and mantissas are stored in separate arrays: 8-bit input exponents (EIs) are allocated to 128 $\times$ 8-bit IECIM arrays, with each row storing one EI, and 8-bit weight exponents (EWs) are allocated to 128 $\times$ 8-bit WECIM arrays. Mantissas are stored in 128 $\times$ 13-bit IMCIM and WMCIM arrays using two's complement format {sign, 1, mantissa, extension-bits}. Both IMCIM and WMCIM contain 128 rows. Each IMCIM row integrates an input sparse feature detection (ISFD) module, while each WMCIM row is equipped with a weight sparse feature detection (WSFD) and an in situ Booth coding unit. The encoder comprises 128 Booth encoders and compensation (BEAC) modules, while the serial accumulator integrates 128 shift serial accumulation (SEA) units. The proposed FP CIM macro performs 128 $\times$ 4 sets of BF16 format MAC operations.

B. Proposed Computation Flow

The BF16 format consists of a 1-bit sign (S), an 8-bit exponent (E), and a 7-bit mantissa (M). The sign bit S

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:57 UTC from IEEE Xplore. Restrictions apply.

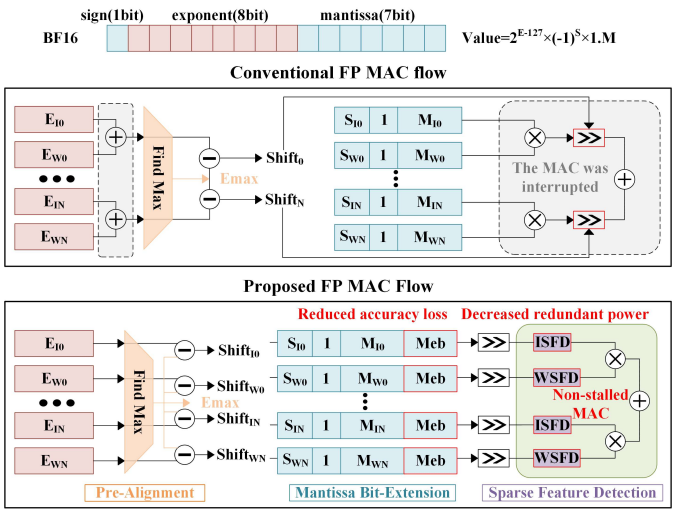


Fig. 2. Comparison of conventional FP MAC and the proposed.

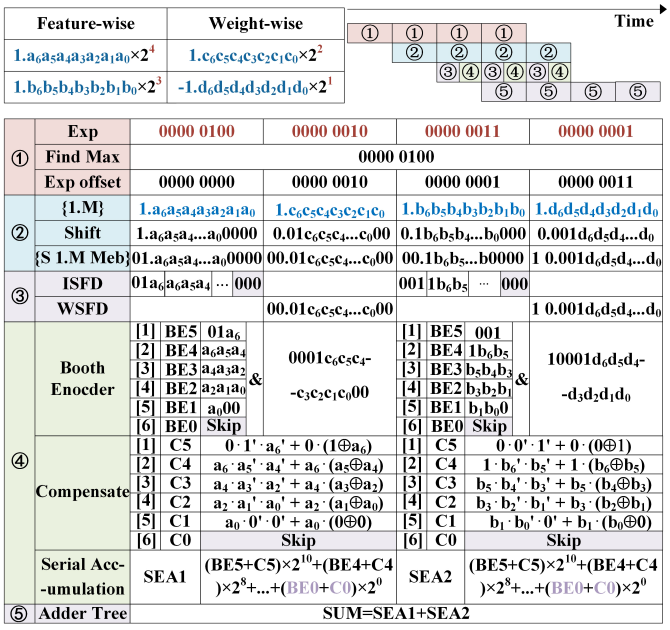


Fig. 3. Pipeline operation of the proposed FP CIM macro.

indicates the sign of the number (0 for positive and 1 for negative). The exponent E is represented in biased notation with a bias of 127. An E value of all ones signifies infinity, and all zeros signifies a denormalized number, resulting in an effective exponent range of −126 (E = 1) to +127 (E = 254) after biasing. To conserve storage bits, the mantissa M omits the leading implicit integer bit "1," storing only the fractional part; however, this hidden bit "1" must be explicitly restored during FP MAC computation. Its value is expressed as Value = $2^{E-127} \times (-1)^S \times 1.M$.

Fig. 2 contrasts the conventional FP MAC flow with the proposed exponent-parallel prealignment and mantissa sparse feature detection computing scheme. The conventional FP MAC process is primarily divided into the exponent and mantissa calculation stages. First, the sum of the exponents (Esums) is obtained by adding the EI and the EW. Next, the maximum value (Emax) among all Esums is identified through comparison. Then, the shift amount (Shift = Emax − Esum) for each exponent is computed. During the mantissa calculation phase, the input mantissa with the restored hidden bit (1.MI) and the weight mantissa (1.MW) are multiplied to yield a product. This product must then be right-shifted by its corresponding shift value to achieve exponent alignment. Finally, the shifted results are accumulated. Crucially, each input-weight pair may have a distinct Esum, requiring an independent shifting alignment before every accumulation. Implementing this frequent and dynamic shifting logic inside the CIM macro introduces significant circuit complexity and undermines architectural integrity. It also incurs substantial area and power overheads.

To address the bottlenecks of the conventional flow, this article proposes an optimized scheme combining exponent-parallel prealignment and mantissa sparse feature detection. First, the EI and EW are compared in parallel to directly identify the global Emax. This step is performed before the mantissa MAC, achieving prealignment of exponents. A critical advantage of this prealignment is that throughout the entire MAC process, the exponents of all intermediate products participating in the accumulation are uniformly fixed at (2Emax − 127). This completely eliminates the need for the dynamic shifting operations required between MAC in the conventional flow due to varying exponent sums. This not only simplifies the computation logic but, more importantly, preserves the inherent coherence and parallelism of the MAC flow in the CIM macro.

While exponent prealignment streamlines the process and enhances efficiency, it may theoretically introduce accuracy loss. The mantissa right-shift during prealignment discards lower significant bits, causing truncation errors. These errors propagate and amplify in subsequent MAC operations. To compensate for this potential loss, the scheme incorporates mantissa extension bits (Meb) during the mantissa calculation phase. This effectively increases the number of significant bits in the mantissa, thereby significantly mitigating the accuracy loss caused by prealignment. Concurrently, to further improve energy efficiency, the scheme performs sparse feature detection on the mantissa before MAC operations of the mantissa. For mantissa identified as sparsity, the system skips the corresponding redundant computation, directly avoiding unnecessary overhead. This mantissa sparse feature detection mechanism effectively reduces energy consumption, particularly beneficial for neural network inference scenarios exhibiting inherent sparsity.

The FP CIM macro's MAC pipeline operation process proposed in this article is shown in Fig. 3, and its core computational steps can be summarized as follows.

① *Exponent-Parallel Prealignment and Shift Control*: EIs and EWs are distributively stored in IECIM and WECIM arrays. Emax is determined through the in-memory parallel comparison mechanism, and the exponent offset of each data is calculated as the benchmark parameter for the shift operation.

② *Mantissa Dynamic Shifting*: Based on the offsets generated in the previous stage, input mantissas and weight mantissas undergo parallel right-shift alignment to ensure a unified exponent (2Emax − 127) for subsequent operations.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:57 UTC from IEEE Xplore. Restrictions apply.

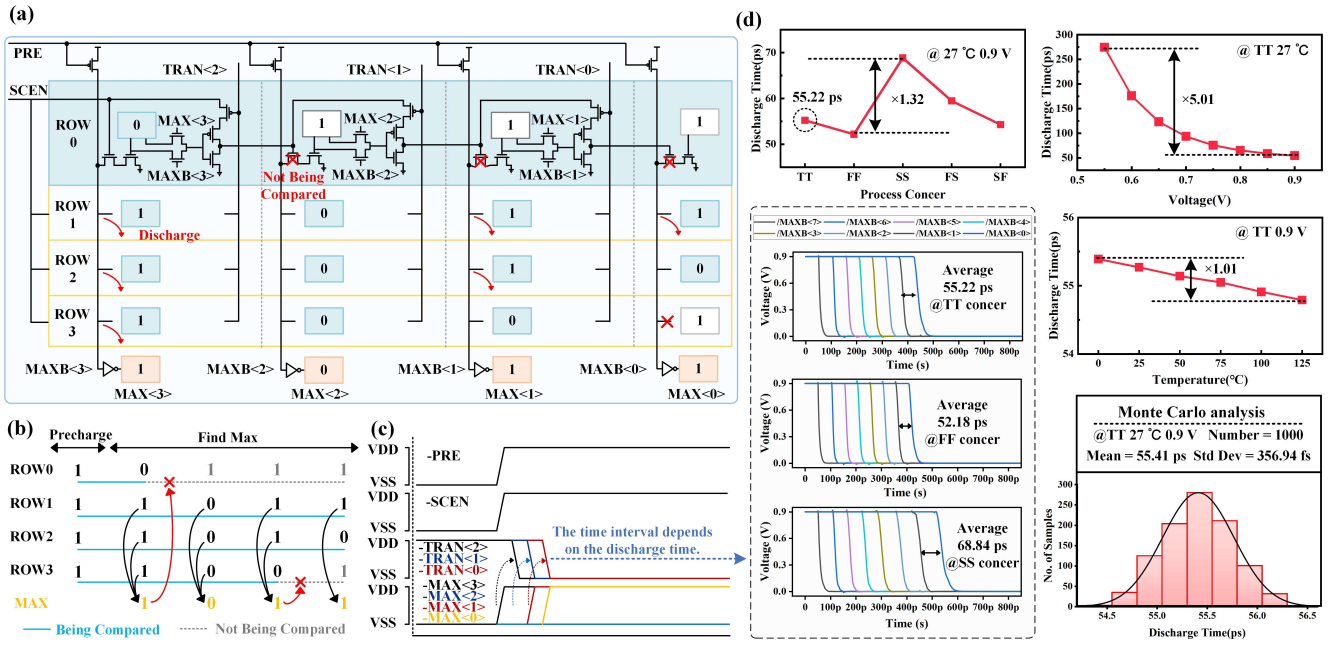


Fig. 4. Proposed in-memory exponent parallel comparison. (a) Schematic of the proposed cascaded row-elimination comparison. (b) Computation flow of finding Emax. (c) Timing diagram of Emax search. (d) PVT and Monte Carlo analysis of the discharge time.

③ *Sparse Feature Detection Cooperative Computation*: For the input mantissa, ISFD cooperates with the encoder, dynamically skipping the corresponding redundant computation steps upon a detected sparsity. For the weight mantissa, the WSFD triggers parallel decisions to gate the participation of subsequent steps.

④ *Coding Compensation and SEA*: Booth-encoded mantissas undergo sign compensation logic for numerical correction, and then the partial product accumulation is realized by adopting the SEA architecture.

⑤ *Multilevel Parallel Accumulation and Quantized Output*: Intermediate results from SEAs are accumulated in parallel through an adder tree. A quantization module finally converts the result to BF16 format, preserving significant mantissa bits and truncating the redundant low-order bits.

C. Maximum Exponent Search

This article proposes an in-memory exponent parallel comparison circuit. All valid exponents are synchronously read out from the memory cells in a single cycle and then undergo bit-level comparisons step by step according to binary weight, from the MSB to the LSB. To ensure the correctness of the comparison process, the circuit incorporates a cascaded row-elimination mechanism. That is, during each bit-level comparison, if the current exponent bit does not reach the maximum value at that bit level, its corresponding subsequent enable path is immediately closed, preventing redundant data from participating in subsequent bit-level comparisons and reducing dynamic power consumption.

To illustrate the operational process of the proposed in-memory exponent parallel comparison circuit more clearly, consider an example with four 4-bit exponents: “0111,” “1011,” “1010,” and “1001,” as shown in Fig. 4(a). The precharge signal PRE is first driven low to precharge the signal MAXB lines of each column to a high level. Subsequently, the start comparison enables signal SCEN to be set high to initiate maximum value comparison for the first column’s stored exponent bits. Since the stored values are “0,” “1,” “1,” and “1,” respectively, the signal MAXB<3> line is discharged to low through memory cells storing “1.” After inversion, this generates a high-level output, establishing the maximum value for this column as MAX<3> = 1. Next, the transmit enable signal TRAN<2> is activated low. Given that MAX<3> = 1 in the first column, the auxiliary circuit disables subsequent comparisons for ROW0. Thus, only rows 1–3 (ROW1, ROW2, and ROW3) participate in subsequent maximum value detection.

Continue to compare the maximum values in the second column. Since the stored information is “0,” “0,” and “0,” respectively, MAXB<2> maintains its precharged high state. Consequently, MAX<2> outputs low, indicating MAX<2> = 0. Then, the comparison of the maximum exponential values in the third column is carried out, and the stored information is “1,” “1,” and “0,” respectively. At the same time, because MAX<1> resolves to “1,” triggering the disablement of ROW3 for subsequent operations. Finally, the fourth column comparison proceeds with bits “1” and “0.” The MAXB<0> line discharges to low, resulting in MAX<0> outputting high (MAX<0> = 1). This sequence conclusively identifies “1011” as the maximum value among the four 4-bit exponents, the operation timing and computation flow are shown in Fig. 4(b) and (c). This demonstrates the circuit’s ability to eliminate noncandidate rows progressively during multibit exponent comparisons.

The sequential setting of TRAN control signals to a low state during the comparison operation is primarily intended to

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:57 UTC from IEEE Xplore. Restrictions apply.

resolve interstage timing conflicts induced by the precharging mechanism. Specifically, during the precharging phase, all column MAXB lines are forcibly initialized to a high-voltage state, resulting in all MAX outputs defaulting to a low logic level. In the absence of stage-isolation control provided by the TRAN signals, when the SCEN signal transitions to high, the first column begins the MSB comparison while the enable signal concurrently propagates to subsequent stages. This premature activation allows invalid exponents to participate in downstream bits comparison, whose discharge activities interfere with the correct voltage determination on subsequent MAXB lines, thereby undermining the validity of the overall comparison results.

In addition, it needs to be emphasized that the interval of TRAN signals is engineered to be exceptionally short. Its duration needs only marginally to exceed the discharge time required for MAXB lines to drop from precharged-high to 0.5 VDD (half-supply voltage). The discharge waveform at the MAXB not only determines the accuracy of the comparison result but also directly impacts circuit performance. The simulation employs an all-"1" input vector pattern (1111_1111) for maximum value comparison. Under this configuration, all eight MAXB signal lines undergo discharge. The discharge interval of each MAXB node was measured, and the average value was used as the key performance metric. This metric has been rigorously validated through multidimensional testing, including simulations across various process, voltage, and temperature (PVT) conditions and Monte Carlo statistical analysis. Detailed simulation results are shown in Fig. 4(d).

Under the TT corner, 27 °C, and 0.9-V supply voltage, the discharge time is 55.22 ps. Regarding process corner variations, the maximum discharge time difference between FF and SS corners is 1.32×. At the TT corner and 27 °C, voltage variations lead to a maximum discharge time that is 5.01 times the minimum, resulting in a difference of 221.43 ps. Conversely, under the TT corner and 0.9 V, temperature variations result in a maximum difference of only 1.01×. Furthermore, 1000-run Monte Carlo simulations at the TT corner, 0.9 V, and 27 °C yielded an average discharge time of 55.41 ps with a standard deviation of 356.94 fs. Simulation-based measurements confirm this duration typically resides in the picosecond scale (10^{-12} s) and exhibits excellent stability under process variations. By controlling the TRAN signals synchronously with picosecond precision, this design eliminates the problem of incorrect comparison result output while fully preserving the parallel architecture advantages of the in-memory exponent parallel comparison circuit, ensuring high computational speed and efficiency.

After comparison, the two's complement of Emax and the exponent values are fed into a full adder chain. This chain performs subtraction through two's complement conversion. The sum bits (S) directly provide the mantissa shift amount, and the stage carry bits (Cout) propagate through the cascaded structure.

D. Booth Encoding and Mantissa Sparse Feature Detection

Radix-4 Booth Encoding, an efficient optimization algorithm for binary multiplication, significantly reduces the computational cycle required for multiplication operations compared to conventional methods by grouping multiplier bits into triplets to generate optimized partial products. Specifically, the partial product computation is expressed as

$$\begin{aligned} \mathrm{IN} \times W &= \sum_{i=0}^{n-1} (\mathrm{IN}[i] \times W \times 2^i) \\ &= \sum_{i=0, i+=2}^{n} ((-2\mathrm{IN}[i+1] + \mathrm{IN}[i] + \mathrm{IN}[i-1]) \times W \times 2^i). \end{aligned} \tag{1}$$

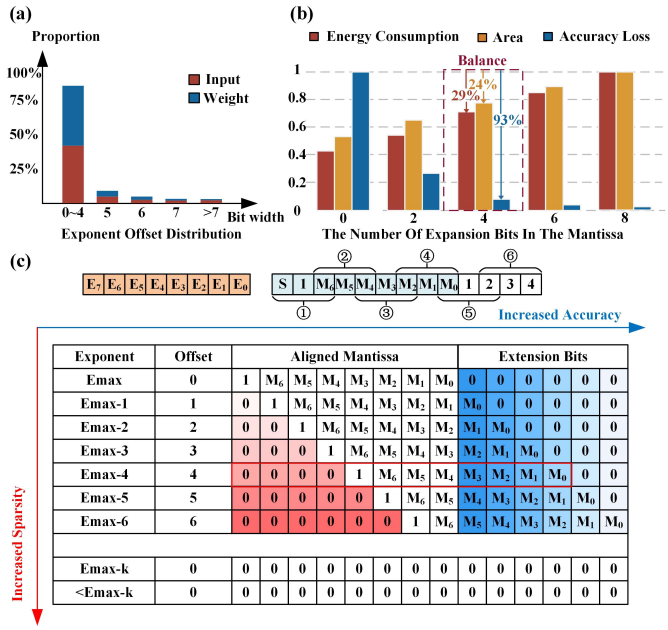
The transformation from the standard form to the Radix-4 Booth-optimized form directly enables the reduction in computation cycles, thereby enhancing energy efficiency. However, current studies overlook the sparsity characteristics in Booth encoding, resulting in redundant multiplicative computations and degraded energy efficiency. The mantissa MAC operations may encounter two sparsity scenarios: ① -2IN[i+1] + IN[i] + IN[i-1] = 0 ② W = 0, either of which collapses the partial product to a sparse zero value. Implementing preencoding sparse feature detection to dynamically bypass redundant encoding and computational steps based on identification outcomes can substantially reduce energy consumption caused by unnecessary operations. Although the adoption of Booth encoding introduces additional area, it effectively halves the number of multiplication cycles, thereby significantly improving the computing speed. Simultaneously, this work innovatively integrates sparse feature detection with Booth encoding, offsetting the area overhead of Booth encoding while exploiting its cycle-reduction advantage and further leveraging sparse feature detection to enhance overall energy efficiency. This co-design mitigates area–speed–energy trade-offs inherent in conventional Booth multipliers.

The truth table of Booth encoding is shown in Fig. 5. This design processes 3-bit input mantissas (IN[i+1:i-1]) and 13-bit weight mantissas (W[12:0]) per cycle according to Booth encoding characteristics. When input mantissas are all-zero (IN[i+1] = 0, IN[i] = 0, IN[i-1] = 0) or all-one (IN[i+1] = 1, IN[i] = 1, IN[i-1] = 1), the ISFD module outputs ALL0 = 1 or ALL1 = 1 to skip subsequent Booth encoding, Booth CIM, and compensation operations. Similarly, when weight mantissas are all-zero (W[12:0] = 0_0000_0000_0000), the WSFD module outputs ALL0 = 1, skipping the same computation flow to save power.

Input			ISFD		Booth Encode		Booth CIM	Compensate
IN[i+1]	IN[i]	IN[i-1]	ALL1	ALL0	NEG	TWO	P[13:0]	C[1:0]
0	0	0	0	1	skip		skip	skip
0	0	1	0	0	0	0	W	0
0	1	0	0	0	0	0	W	0
0	1	1	0	0	0	1	2W	0
1	0	0	0	0	1	1	-2W	2
1	0	1	0	0	1	0	-W	1
1	1	0	0	0	1	0	-W	1
1	1	1	1	0	skip		skip	skip
Weight			WSFD		Booth Encode		Booth CIM	Compensate
W[12:0]			ALL0		NEG	TWO	P[13:0]	C[1:0]
0 0000 0000 0000			1		skip		skip	skip
others			0		--	--	--	--

Fig. 5. Two types of sparse conditions and truth tables for Booth encoding.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:57 UTC from IEEE Xplore. Restrictions apply.



Exponent	Offset	Aligned Mantissa								Extension Bits					
Emax	0	1	M_6	M_5	M_4	M_3	M_2	M_1	M_0	0	0	0	0	0	0
Emax-1	1	0	1	M_6	M_5	M_4	M_3	M_2	M_1	M_0	0	0	0	0	0
Emax-2	2	0	0	1	M_6	M_5	M_4	M_3	M_2	M_1	M_0	0	0	0	0
Emax-3	3	0	0	0	1	M_6	M_5	M_4	M_3	M_2	M_1	M_0	0	0	0
Emax-4	4	0	0	0	0	1	M_6	M_5	M_4	M_3	M_2	M_1	M_0	0	0
Emax-5	5	0	0	0	0	0	1	M_6	M_5	M_4	M_3	M_2	M_1	M_0	0
Emax-6	6	0	0	0	0	0	0	1	M_6	M_5	M_4	M_3	M_2	M_1	M_0
Emax-k	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
<Emax-k	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Fig. 6. Mantissa bit-extension strategy for BF16 computation. (a) Statistical distribution of the input and EW offsets. (b) Tradeoffs among energy consumption, area, and accuracy loss. (c) Effects of exponent offset and mantissa extension bits on sparsity and accuracy.

The sparse feature detection is executed after mantissa shifting but before multiplication. Fig. 6(a) shows the exponent offset distribution (Emax minus input/EW) for BF16 data during EfficientNet-B0 inference. It can be concluded that approximately 85.6% of offsets fall within the 0–4 bit range, forming the dominant region, while higher offsets (5/6/7 bits and > 7 bits) decrease progressively [14]. Crucially, larger exponent offsets correspond to smaller mantissa values. At this time, the truncation operation on the mantissa has an extremely limited impact on the final calculation accuracy, thereby achieving an effective tradeoff between computational efficiency and accuracy.

Meanwhile, mantissa shifting with LSB zero-padding substantially increases sparsity. Combined with the tradeoff of energy consumption, area, and accuracy loss shown in Fig. 6(b), analysis of exponent offsets and mantissa bit-extension width impacts shown in Fig. 6(c) reveals that mantissa sparsity increases with larger exponent offsets, While wider extension bits reduce computational accuracy loss, they also increase power consumption and area overhead. For the dominant 0–4-bit offset range (covering 85.6% of cases), a 4-bit mantissa bit-extension achieves optimal balance with < 7% accuracy loss at minimal energy consumption and area overhead. Consequently, the original 7-bit mantissa of BF16 is extended to 11 bits. Including sign and hidden bits, the MAC operation uses a 13-bit width, requiring exactly six cycles for Booth encoding.

E. Mantissa Booth Encoding

Fig. 7(a) details the corresponding circuit implementation of the IMCIM, WMCIM, and the encoder. To achieve parallel encoding computation of the mantissas, each 13-bit input mantissa and its corresponding 13-bit weight mantissa share the same BEAC. For clarity in explaining the operational principle, the working process of this circuit is divided into the following phases.

1) Mantissa sparse feature detection. The IMCIM and WMCIM concurrently perform sparse feature detection on input mantissas and weight mantissas. Since the Booth encoder processes input mantissas in 3-bit sliding windows but encodes weight mantissas at full-bitwidth, the 13-bit input mantissa detection requires six clock cycles, whereas the 13-bit weight mantissa is processed within a single cycle. In each processing cycle, ISFD and WSFD perform sparse feature detection on the mantissa in parallel. When a mantissa with sparse features is detected, the generated input sparse indication signal SPIN and the weight sparse indication signal SPW are used to regulate the operating state of BEAC, enabling or disabling it. To ensure computational continuity, upon completing partial input mantissa detection each cycle, the current 3-bit window data (IN[i+1], IN[i], IN[i-1]) is immediately routed to the BEAC module for Booth encoding and compensation operations. At the circuit level, the IMCIM array is composed of an alternating cascade of SRAM cells C1 and C2, while the WMCIM array employs a C3-cell cascade structure. The circuit structures of C1, C2, and C3 are shown in Fig. 7(b). This implementation simultaneously supports both sparse feature detection and encoded data readout functionalities.

2) Booth encoding and compensation. Since the mantissa computation employs two's complement, the MSB of negative values inherently carries a negative weight that is not fully captured in standard Booth-encoded partial products. To rectify the computational deviation caused by this arithmetic discrepancy, a compensation circuit is cascaded concurrently with the Booth encoder. This circuit applies weight-adjusted correction to partial products, guaranteeing arithmetic accuracy compliant with two's complement rules. During the mantissa Booth encoding and compensation phase, the computation path is dynamically regulated based on sparsity indication signals SPIN and SPW. A high SPW (logic "1") triggers global bypass of all computation cycles; when SPW is low but SPIN is high ("1"), only the Booth encoding and BEAC compensation of the current cycle are skipped; with both signals low ("0"), the Booth encoder activates normally to generate the bit-inversion control (NEG), doubling control (TWO), and the compensation circuit generates 2-bit compensation value (C[1:0]). This hierarchical bypass mechanism achieves dynamic hardware gating through direct signal control, substantially reducing redundant power consumption under sparse mantissa conditions.

3) Weight mantissa coding in memory. The sparse indication signals SPIN and SPW jointly control the coding. High state ("1") on either signal skips weight mantissa coding; only when both are low ("0") is the SRAM-based Booth-coding multiplier activated. This architecture implements in-memory computation through a four-transistor XOR gate. The inversion signal NEG and storage node Q of SRAM cell C3 serve as dual inputs, with the XOR output driving a shift selector controlled by the doubling signal TWO and selecting

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:57 UTC from IEEE Xplore. Restrictions apply.

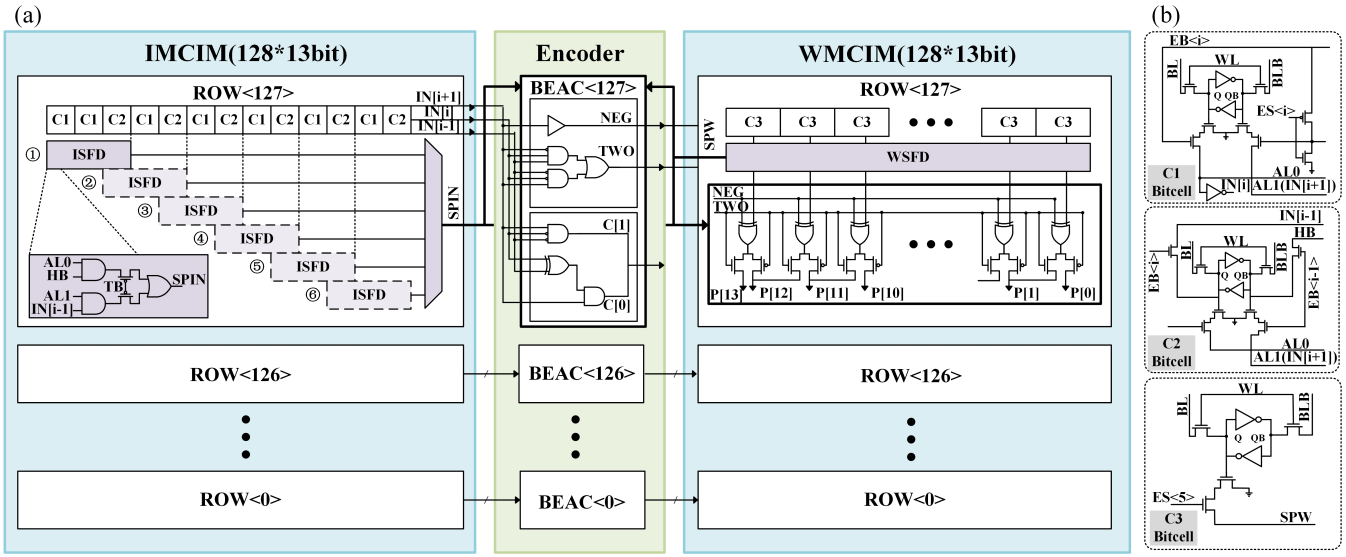


Fig. 7. Structure of the mantissa processing module. (a) Schematic of the proposed mantissa Booth encoding. (b) SRAM bitcells that constitute the mantissa operation architecture.

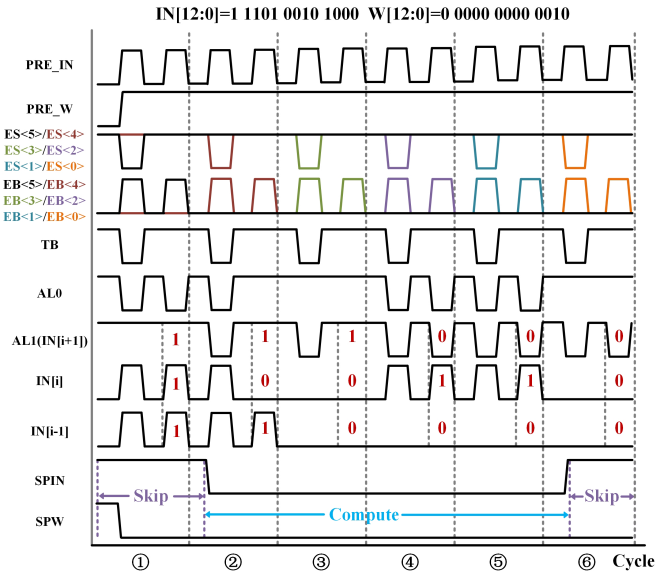


Fig. 8. Example timing waveform of mantissa sparse feature detection and Booth encoding skipping redundant operation.

left-shifted output when TWO = 1 or direct output otherwise, ultimately generating the 14-bit partial product P[13:0].

Fig. 8 illustrates an example waveform of the proposed IMCIM and WMCIM, taking the input mantissa IN[12:0] = 1_1101_0010_1000 and the weight mantissa W[12:0] = 0_0000_0000_0010 as an example. The circuit executes six-cycle sparse feature detection and Booth-encoded data readout following the Booth encoding protocol. ① During the initial phase, precharge signals PRE_IN and PRE_W are driven low to precharge nodes AL0, AL1, SPIN, and SPW to high logic levels. Subsequently, enabling signal ES<5> turns low while EB<5> goes high, activating sparse feature detection for the 3-bit input mantissa segment of IN[12:10] and full weight mantissa W[12:0]. When the signal TB is asserted low, SPIN and SPW are generated for the current cycle. Given IN[12:10] = 111 (all-ones) and a nonzero W[12:0] in this example, SPIN remains high, whereas SPW discharges to low. Following re-precharging of AL0/AL1 to high, the sensed 3-bit input IN[12:10] is read out as IN[i+1] = 1, IN[i] = 1, and IN[i-1] = 1 for Booth encoding and compensation operations. The high SPIN flag (indicating input sparsity) triggers bypassing of subsequent operations in this cycle. ②–⑥ The identical procedure is applied to the remaining data segments, such as IN[10:8] = 101, where normal computation proceeds when nonsparsity is detected. This cycle-based operation repeats across six cycles until traversal of IN[12:0] is achieved in sequence.

Sparsity is determined by specific bit patterns. SPIN is asserted high when input mantissa segments equal all-ones (111) or all-zeros (000), while SPW flags high exclusively for all-zero weight mantissa (0_0000_0000_0000). This preencoding sparse feature detection mechanism effectively skips redundant computations, achieving significant energy efficiency improvement.

F. Mantissa Partial Product Accumulation

Fig. 9 shows the structure of the mantissa partial product accumulation module. The Booth encoding processes 3-bit segments of a 13-bit input mantissa per clock cycle, requiring six cycles for complete computation. Each cycle generates a 14-bit partial product P[13:0] and a 2-bit compensation value C[1:0]. The full-precision product of the 13-bit input mantissas and weight mantissas is obtained through bit-level shifted accumulation of these outputs. To prevent accuracy loss from early-stage truncation, a 14-bit adder integrates P[13:0] and C[1:0] in each cycle. The summed results are then left-shifted by 10, 8, 6, 4, 2, 0 bits, respectively, implementing the $\times 2i$ operation inherent to Booth encoding. These shifted outputs are serially accumulated in a 24-bit adder over six cycles, generating a 24-bit product sum.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:57 UTC from IEEE Xplore. Restrictions apply.

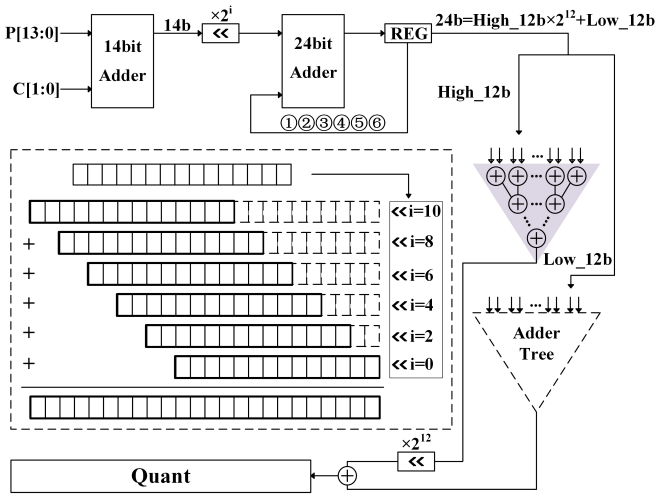


Fig. 9. Structure of serial partial product accumulation for mantissa and the time-multiplexed 12-bit high/low adder tree.

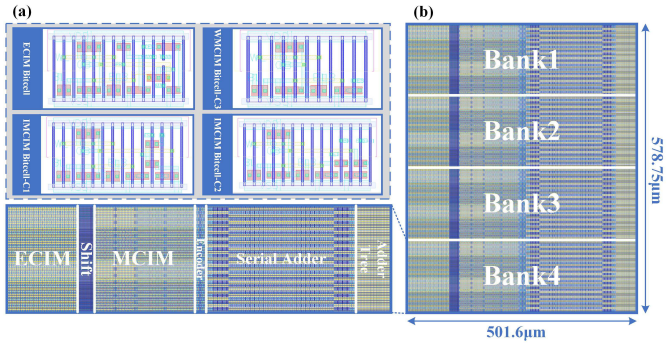


Fig. 10. Partial layout of the proposed FP CIM. (a) Layouts of the 12T SRAM cell for the exponent array and the cells (C1–C3) for the mantissa array. (b) Layout of the FP CIM macro.

For parallel accumulation of multiple 24-bit products, a high/low 12-bit time-multiplexed adder tree architecture is proposed. Higher 12-bit segments are first processed through a Wallace adder tree, followed by reusing the same adder structure for lower 12-bit segments. The time-multiplexed scheme reduces hardware resources by 45% relative to a conventional 24-bit Wallace tree. By avoiding the latency scaling of carry save adder architectures, the proposed structure reduces critical path delay by 51%. This optimization stems from halving the operational bit-width and reusing accumulation logic. In the final stage, the higher 12-bit result is shifted left by 12 bits, added to the lower 12-bit result, and the output is normalized to generate a standardized product.

III. Simulation and Analysis

The proposed FP CIM macro is realized with a 28-nm CMOS process, and it supports BF16 input/weight MAC operations. Based on the design utilizing both standard cells and custom cells, the detailed layouts of the critical custom SRAM cells enabling this functionality are illustrated, including the specialized 12T cell for the exponent array and the C1/C2/C3 cells for the mantissa array [Fig. 10(a)]. Furthermore, the comprehensive layout of the entire macro, occupying an area of 501.6 × 578.75 μm, is presented in Fig. 10(b). The area breakdown of the proposed macro is shown in Fig. 11(a). The serial adder, owing to its implementation of full-precision partial product accumulation, occupies a relatively large area share. Additionally, the MCIM and ECIM modules also contribute significantly to the area.

As shown in the power breakdown of Fig. 11(b), the power consumption of this macro is 1.73 mW, among which the ECIM and MCIM modules dominate. Compared to traditional architectures where the mantissa computation consumes over 70% of the power, the proposed architecture proposes mantissa in-memory Booth computing and sparse feature detection to enhance energy efficiency in mantissa computation. This approach ultimately achieves a balanced power distribution between exponent and mantissa.

Fig. 11(c) shows the voltage–frequency scaling with power measurement results of the proposed FP CIM macro. It operates at a frequency range of 66–496 MHz under a supply voltage of 0.55–0.9 V. The macro achieves a maximum frequency of 496 MHz at a supply voltage of 0.9 V. At 0.6 V and 105 MHz, the peak energy efficiency reaches 12.88 TFLOPS/W.

Fig. 11(d) shows the operation cycles and power consumption of MCIM and BEAC modules at different mantissa sparsity levels. Sparsity is defined as the proportion of "111" or "000" in input mantissa data under radix-4 Booth encoding. Increased sparsity proportionally reduces computation cycles and power consumption by enabling dynamic skipping of redundant encoding/compensation steps through sparse feature detection. At maximum sparsity, this achieves 50% fewer operation cycles and 83% lower power consumption compared to the zero-sparsity scenario.

Fig. 11(e) and (f) illustrates the variations in energy efficiency of the proposed design in relation to data sparsity and toggle rate. The results reveal that energy efficiency exhibits a strong positive correlation with data sparsity. This is attributed to the dynamic skipping of redundant computations through the sparse feature detection mechanism, achieving up to 2.52× energy efficiency improvement at 0.6 V. Conversely, energy efficiency shows a negative correlation with the toggle rate. Reducing the toggle rate significantly enhances energy efficiency, with a 20% toggle rate delivering 5.15× higher energy efficiency than the 100% case at 0.6 V. These results quantitatively delineate the critical impact of workload characteristics on energy efficiency, validating the effectiveness of the design in processing highly sparse data.

Based on cycle comparisons of exponent/mantissa operations for 8 BF16 MAC operations shown in Fig. 11(g). The proposed parallel exponent comparison reduces exponent processing cycles by up to 87%, while the optimized mantissa encoding architecture achieves up to 65% cycle reduction. Notably, cycles in [14], [16] and [17]'s exponent structures increase with larger data volumes. Consequently, the proposed architecture exhibits cycle efficiency in large-scale data-parallel computing. This macro combines two key optimizations: 1) increased parallelism in exponent comparison with sparse feature detection to eliminate redundant mantissa

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:57 UTC from IEEE Xplore. Restrictions apply.

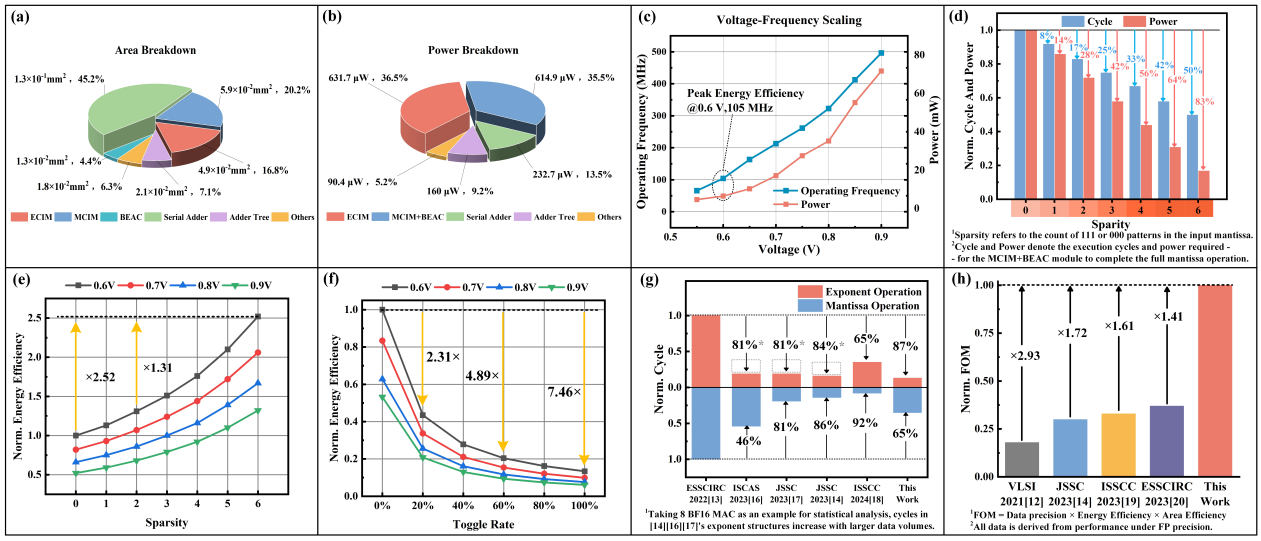


Fig. 11. Performance simulation and analysis. (a) Area breakdown. (b) Power breakdown. (c) Voltage–frequency scaling with power measurement results. (d) Operation cycles and power across different sparsity. (e) Energy efficiency under different sparsity. (f) Energy efficiency under different toggle rates. (g) Cycle comparisons of exponent/mantissa operations. (h) FOM scores under various CIM structures.

TABLE I
COMPARISON WITH PRIOR FP CIM WORKS

	VLSI 2021 [12]	ESSCIRC 2022 [13]	ISCAS 2023 [16]	ISSCC 2023 [19]	ESSCIRC 2023 [20]	TCASII 2023 [21]	This Work
Exponent Structure	Digital CIM	Digital CIM	Digital CIM	Analog CIM	Digital CIM	Analog CIM	Digital CIM
Mantissa Structure	Digital	Digital CIM	Digital CIM	Hybrid CIM	Digital CIM	Analog CIM	Digital CIM
Technology(nm)	28	28	40	22	28	40	28
Macro Size(kb)	396	16	12	832	4	32	19
Supply Voltages(V)	0.68-1.1	0.5-0.9	0.7-1.1	0.6-0.8	0.55-1.2	1.1	0.55-0.9
Precision[1]	BF16	FP8-FP32	BF16/FP32	BF16	FP8/BF8	FP8/bFP16	BF16
Sparsity Support[2]	No	No	No	Yes	No	No	Yes
Marco Area(mm^2)	5.83	0.0439	0.48	-	0.71	0.29	0.29
Memory Density(kb/mm^2)	67.92	364.46	25	-	5.63	110.34	65.12
Operating Frequency(MHz)	250	53-403	53-294	151-156	650/0.0024	-	66-496
Peak Energy Efficiency (TFLOPS/W)[3]	1.43	1.64-0.099	1.97/0.62	70.21	12.1/66.6	8.8/3.3	12.88
Peak Area Efficiency ($TFLOPS/mm^2$)[3]	0.21	0.058-0.005	-	0.071	0.94	-	2.27

1. Statistics exclusively include floating-point precision but integer precision is not encompassed.
2. Whether algorithmic or circuit optimizations for floating-point sparsity are implemented in this work.
3. Maximum reported value in references [12],[13],[16],[19]-[21].

operations and 2) bit-extension based area–accuracy–energy co-optimization. Together, these achieve up to 5.59× improvement in FOM, as shown in Fig. 11(h). To strengthen system-level validation, we evaluated our design with ResNet-32 on CIFAR-10, achieving 93.1% accuracy. This demonstrates significant improvement in computational precision while maintaining compatibility.

Table I compares the proposed FP CIM macro with previous work [12], [13], [16], [19], [20], [21]. Through optimized design of the exponent comparison and mantissa multiplication modules, the proposed macro achieves a maximum operating frequency of 496 MHz. The parallel exponent comparison mechanism enhances computational parallelism, while the mantissa sparse feature detection reduces power consumption. Ultimately, the architecture demonstrates improved area efficiency of 2.27 TFLOPS/mm^2 and energy efficiency of 12.88 TFLOPS/W relative to the advanced FP CIM schemes presented in the table.

IV. CONCLUSION

This work proposes a 28-nm FP CIM macro to enhance energy efficiency for high-accuracy AI computing. To address limited exponent-path parallelism, a cascaded row-elimination comparison mechanism is proposed. To overcome low mantissa-path energy efficiency, an in-memory sparse feature detection method is introduced to skip redundant computations. The separated exponent/mantissa CIM pipeline with mantissa bit-extension further optimizes accuracy, area, and energy efficiency of the proposed macro. Simulation results show the 19-kb macro achieves 496 MHz at 0.9 V with 12.88 TFLOPS/W energy efficiency, providing an efficient CIM solution for high-accuracy neural networks.

REFERENCES

[1] X. Si et al., "A 28 nm 64 Kb 6T SRAM computing-in-memory macro with 8b MAC operation for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2020, pp. 246–248.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:57 UTC from IEEE Xplore. Restrictions apply.

[2] J.-W. Su et al., "A 28 nm 64 Kb inference-training two-way transpose multibit 6T SRAM compute-in-memory macro for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2020, pp. 240–242.
[3] J.-W. Su et al., "A 28 nm 384 kb 6T-SRAM computation-in-memory macro with 8b precision for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2021, pp. 250–252.
[4] J.-M. Hung et al., "An 8-Mb DC-current-free binary-to-8b precision ReRAM nonvolatile computing-in-memory macro using time-space-readout with 1286.4–21.6TOPS/W for edge-AI devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2022, pp. 1–3.
[5] C.-X. Xue et al., "15.4 A 22 nm 2Mb ReRAM compute-in-memory macro with 121–28TOPS/W for multibit MAC computing for tiny AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 244–246.
[6] C.-X. Xue et al., "A 22 nm 4 Mb 8b-precision ReRAM computing-in-memory macro with 11.91 to 195.7 TOPS/W for tiny AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 245–247.
[7] B. Wang et al., "A 28 nm horizontal-weight-shift and vertical-feature-shift-based separate-WL 6T-SRAM computation-in-memory unit-macro for edge depthwise neural-networks," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 134–136.
[8] P. Chen et al., "A 22 nm delta-sigma computing-in-memory (16CIM) SRAM macro with near-zero-mean outputs and LSB-first ADCs achieving 21.38 TOPS/W for 8 b-MAC edge AI processing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 140–142.
[9] P.-C. Wu et al., "A 28 nm 1Mb time-domain computing-in-memory 6T-SRAM macro with a 6.6ns latency, 1241GOPS and 37.01TOPS/W for 8b-MAC operations for edge-AI devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 1–3.
[10] Y.-D. Chih et al., "An 89 TOPS/W and 16.3 TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22 nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 252–254.
[11] H. Mori et al., "A 4 nm 6163-TOPS/W/b 4790 TOPS/mm^2/b SRAM based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous MAC and weight update," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 132–134.
[12] J. Lee et al., "A 13.7 TFLOPS/W floating-point DNN processor using heterogeneous computing architecture with exponent-computing-in-memory," in *Proc. Symp. VLSI Circuits*, Jun. 2021, pp. 1–2.
[13] S. Jeong, J. Park, and D. Jeon, "A 28 nm 1.644 TFLOPS/W floating-point computation SRAM macro with variable precision for deep neural network inference and training," in *Proc. IEEE 48th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2022, pp. 145–148.
[14] F. Tu et al., "ReDCIM: Reconfigurable digital computing- in -memory processor with unified FP/INT pipeline for cloud AI acceleration," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 243–255, Jan. 2023.
[15] S. Yan et al., "A 28-nm floating-point computing-in-memory processor using intensive-CIM sparse-digital architecture," *IEEE J. Solid-State Circuits*, vol. 59, no. 8, pp. 2630–2643, Aug. 2024.
[16] Y. Mai, M. Wang, C. Zhang, B. Zhong, and Z. Yu, "A 1.97 TFLOPS/W configurable SRAM-based floating-point computation-in-memory macro for energy-efficient AI chips," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Monterey, CA, USA, May 2023, pp. 1–5.
[17] A. Guo et al., "A 28-nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," *IEEE J. Solid-State Circuits*, vol. 59, no. 9, pp. 3032–3044, Sep. 2024.
[18] R. Guo et al., "20.2 A 28 nm 74.34TFLOPS/W BF16 heterogenous CIM-based accelerator exploiting denoising-similarity for diffusion models," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 362–364.
[19] P.-C. Wu et al., "A 22 nm 832 kb HybridDomain floating-point SRAM in-memory-compute macro with 16.2–70.2 TFLOPS/W for high-accuracy AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 126–128.
[20] J. Saikia, A. Sridharan, I. Yeo, S. Venkataramanaiah, D. Fan, and J.-S. Seo, "FP-IMC: A 28 nm all-digital configurable floating-point in-memory computing macro," in *Proc. IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2023, pp. 405–408.
[21] X. Hu et al., "An 8.8 TFLOPS/W floating-point RRAM-based compute-in-memory macro using low latency triangle-style mantissa multiplication," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 11, pp. 4216–4220, Nov. 2023.



Wenjuan Lu (Member, IEEE) received the M.S. degree in circuits and systems and the Ph.D. degree in microelectronics and solid-state electronics from Anhui University, Hefei, China, in 2014 and 2017, respectively.

She is currently an Associate Professor of Microelectronics and Solid-State Electronics with Anhui University. Her current research interests include next-generation nonvolatile memory and the circuit design of low-power, high-performance, and high-reliability memory.



Hao Liu received the B.S. degree in microelectronic science and engineering from Anhui University, Hefei, China, in 2023, where he is currently working toward the M.S. degree in integrated circuit science and engineering.

His research interests include floating-point computing and SRAM-based computing-in-memory architecture design.



Kang Meng received the B.S. degree in microelectronic science and engineering from Anhui University, Hefei, China, in 2023, where he is currently working toward the M.S. degree in integrated circuit engineering.

His current research interests include static random access memory and in-memory computing circuits.



Xiaobo Gong received the B.S. degree in photonic information science and engineering from Yangtze University, Jingzhou, China, in 2021. He is currently working toward the M.S. degree in integrated circuit engineering at Anhui University, Hefei, China.

His research interests mainly focus on static random access memory and computing-in-memory circuits.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:57 UTC from IEEE Xplore. Restrictions apply.



Zhiqiang Wang received the B.S. degree in electronic and information engineering from Huaibei Normal University, Huaibei, China, in 2018. He is currently working toward the M.S. degree in microelectronics and solid-state electronics at Anhui University, Hefei, China.

His research interests mainly focus on static random access memory and computing-in-memory circuits.



Wei Hu received the Ph.D. degree from the University of Science and Technology of China, Hefei, China, in 2021.

She is currently a Lecturer with the School of Integrated Circuits, Anhui University, Hefei. Her research directions include SRAM memory computing circuits and the design of the next-generation nonvolatile ferroelectric memory computing circuits.



Chunyu Peng (Member, IEEE) received the M.S. degree in circuits and systems and the Ph.D. degree in microelectronics and solid-state electronics from Anhui University, Hefei, China, in 2013 and 2022, respectively.

He is currently an Associate Professor of Microelectronics and Solid-State Electronics with Anhui University. His research interests include signal processing, analog IC design, and high-performance, reliable memory technology.



Shan Gao received the M.S. and Ph.D. degrees in circuits and systems from Anhui University, Hefei, China, in 2004 and 2007, respectively.

She is currently a Lecturer with Anhui University, specializing in advanced microelectronics device research. Her primary focus lies in power semiconductor devices, tunnel field-effect transistors, and quantum-effect-based novel device architectures.



Zhiting Lin (Senior Member, IEEE) received the B.S. and Ph.D. degrees in electronics and information engineering from the University of Science and Technology of China (USTC), Hefei, China, in 2004 and 2010, respectively.

From 2015 to 2016, he was a Visiting Scholar with the Engineering and Computer Science Department, Baylor University, Waco, TX, USA. In 2011, he joined the School of Integrated Circuits, Anhui University, Hefei, Anhui, where he is currently a Professor. He has published about 50 articles and holds over 20 Chinese patents. His research interests include pipeline analog-to-digital converters and high-performance static random access memory. His current research interests include low-power circuit design, computing-in-memory, and nonvolatile memory and system design.



Xiulong Wu (Senior Member, IEEE) received the B.S. degree in computer science from the University of Science and Technology of China (USTC), Hefei, China, in 2001, and the M.S. and Ph.D. degrees in electronic engineering from Anhui University, Hefei, in 2005 and 2008, respectively.

From 2013 to 2014, he was a Visiting Scholar with the Engineering Department, The University of Texas at Dallas, Richardson, TX, USA. He is currently a Professor with Anhui University. He has published about 60 articles and holds over ten Chinese patents. His research interests include high-performance static random access memory and mixed-signal ICs. His current research interests include low-power circuit design, computing-in-memory, and nonvolatile memory and system design.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:35:57 UTC from IEEE Xplore. Restrictions apply.